

```

found=99;
do
{
    j++;
    if(j%newline_charnum == 0 && j!=0)
        fscanf(fdi, "\n");
    found=fscanf(fdi, "%f", &x_f[sample_ptr][j]);
}
while(found!=EOF && j<=no_inputs-1);

    actual_no_inputs=j-1;
    for(i=0;i<=actual_no_inputs;i++)
    {
        if(i%5==0) printf("\n");
        printf("%f", x_f[sample_ptr][i]);
        if(x_f[sample_ptr][i]==0)
            x_f[sample_ptr][i]=-1;
    }
fclose(fdi);

    for(i=0;i<=actual_no_inputs;i++)

        sum+=(x_f[sample_ptr][i]*x_f[sample_ptr][i]);

    for(i=0;i<=actual_no_inputs;i++)
        x_f[sample_ptr][i]=x_f[sample_ptr][i]/sqrt(sum);
}
calculate_actual_output()
{
    float sum, input;
    largest=-999;
    for(j=0;j<kohonen_nodes;j++){
        sum=0.0;
        for(i=0;i<=no_inputs;i++)
            sum += w1[j][i]*x1[i];
        if(sum>largest)
        {
            largest=sum;
            number =j;
        }
    }
    // printf("neuron=%d,sum=%f\n",j,sum);
    printf(".. ");
    // delay(10);
}
for(j=0; j<kohonen_nodes;j++)

```



```

x2[j]=0.0;
x2[number]=1.0;
for(j=0;j<kohonen_nodes;j++)
    if(x2[j]==1)
        printf(".. ");
// delay(10);
// printf("winning kohonen =%d\n",j);

for(j=0;j<grossberg_nodes;j++) {
    sum=0.0;
    for(i=0;i<kohonen_nodes;i++)
        sum += w2[j][i]*x2[i];
    x3[j]=sum;
}
for(j=0;j<grossberg_nodes;j++)
    printf(".. ");
// delay(10);
// printf("\ngrossberg = %d %f",j,x3[j]);
// printf("\n_____ \n");
}

```

recalculate_weight_matrix()

```

{
    float sum;
    if(learn_ptr==increment +1){
        counter += 0.0;
        count_beta += 0.0;
    }
    else
    {
        counter+=0.02;
        count_beta+=0.02;
    }
}

```

train_rate_coef= 0.7 * exp(-counter);

```

for(j=0;j<kohonen_nodes;j++)
    for(i=0;i<no_inputs;i++){
        if(j==number)
            w1[j][i]=w1[j][i]+ (train_rate_coef*(x1[i]-w1[j][i]));
    }

```

sum=0.0;


```

for(i=0;i<no_inputs;i++)
sum += (w1[number][i]*w1[number][i]);
if( sum > 0.0 ){
for(i=0;i<no_inputs;i++)
w1[number][i]=w1[number][i]/sqrt(sum);
}

beta=.1*exp(-count_beta);
for(j=0;j<grossberg_nodes;j++)
for(i=0;i<kohonen_nodes;i++) {
if(i==number){
w2[j][i]=w2[j][i]+(beta*desired[j]-w2[j][i]);
}
}
increment=learn_ptr;
}

```

initialize_values()

```

{
float sum;

srand(2);
for(j=0; j<kohonen_nodes; j++)
for(i=0; i<no_inputs; i++)
w1[j][i]=(rand()/32676.0 -.5)*1.0;
srand(2);
for(j=0; j<grossberg_nodes; j++)
for(i=0; i<kohonen_nodes; i++)
w2[j][i]=(rand()/32676.0 -.5)*1.0;
sum = 0.0;
for(j=0; j<kohonen_nodes; j++)
for(i=0; i<no_inputs; i++)
sum += (w1[j][i] * w1[j][i]);

for(j=0; j<kohonen_nodes; j++)
for(i=0; i<no_inputs; i++)
w1[j][i] = w1[j][i]/sqrt(sum);

sum = 0.0;
for(j=0; j<grossberg_nodes; j++)
for(i=0; i<kohonen_nodes; i++)
sum += (w2[j][i] * w2[j][i]);

for(j=0; j<grossberg_nodes; j++)
for(i=0; i<kohonen_nodes; i++)
w2[j][i] = w2[j][i]/sqrt(sum);
}

```



```

for(i=0; i<grossberg_nodes; i++)
desired[i]=0.0;

}

```

```

train()

```

```

{
    int total_no_trials;
    while (train_rate_coef > 0.01)
    {
        for(learn_ptr=0;learn_ptr<n_learn_set;learn_ptr++){
// printf("learn_ptr=%d, training rate =%f\n",learn_ptr,train_rate_coef);
// printf("beta rate = %f\n",beta);

```

```

        for(i=0;i<n_learn_set;i++)
        desired[i]=0.0;
        desired[learn_ptr]=1.0;

```

```

        for(i=0;i<no_inputs;i++)
        x1[i]=x_f[learn_ptr][i];
        calculate_actual_output();
        recalculate_weight_matrix();

```

```

    }

```

```

}

```

```

i=999;

```

```

}

```

```

test_network()

```

```

{

```

```

    int test_ptr;
    FILE *fdi;
    int found;
    float sum;
    char test_file[20];
    cleardevice();
    // clrscr();

```

```

        for(test_ptr=0;test_ptr<n_test_set;test_ptr++)

```

```

        {
            settextstyle(1,0,1);
            outtextxy(100,42," PLEASE ENTER THE TEST FILE NAME : ");
            // printf("PLEASE ENTER THE TEST FILE NAME :=> ");
            gotoxy(60,4);
            scanf("%s",&test_file);
            fdi=fopen(test_file,"r");

```



```

        fdi=fopen(test_file,"r");
        printf("\n Reading file ==>%s\n",test_file);
        if(fdi==NULL)
        {
            printf("could not open input file!\n");
            printf("press any key to continue.");
            fclose(fdi);
            i=-999;
            return(1);
        }
        printf("\n\nReading the data....");
        j=-1;
        found=999;
do
{
    j++;
    if(j%newline_charnum==0 && j!=0)
        fscanf(fdi,"%n");
    found=fscanf(fdi,"%f",&x_f[test_ptr][j]);
} while (found!=EOF && j<=no_inputs-1);

    actual_no_inputs=j-1;
    for(i=0;i<=actual_no_inputs;i++)
    {
        if(i%5==0) printf("\n");
        printf("%f",x_f[test_ptr][i]);
        if(x_f[test_ptr][i]==0)
            x_f[test_ptr][i]=-1;
    }
    fclose(fdi);
    getch();

    sum=0.0;
    for(i=0;i<=actual_no_inputs;i++)
        sum+=(x_f[test_ptr][i] * x_f[test_ptr][i]);

    if(sum>0.0)
    {
        for(i=0;i<=actual_no_inputs;i++)
            x_f[test_ptr][i]=x_f[test_ptr][i]/sqrt(sum);
    }
    for(i=0;i<no_inputs;i++)
        x1[i]=x_f[test_ptr][i];
    printf("\n\n");
    calculate_actual_output();
    outtextxy(300,400," PRESS ANY KEY TO CONTINUE... ");

```



```

getch();
}
i=999;
}

```

```

void read_weights()
{

```

```

    static char weightf[]="weights.wts";
    FILE *fdi;
    int found = 99;
    char temp2[80];
    float sum;
    clrscr();
    cleardevice();
    fdi=fopen(weightf,"r");
    printf("reading file==>%s\n",weightf);
    getch();
    if(fdi==NULL)
    {
        printf("could not open input file!\n");
        printf("press any key to continue.");
        fclose(fdi);
        i=-999;
        return;
    }
    printf("\n\n reading the weights....\n\n");
    getch();

```

```

    for(i=0;i<kohonen_nodes;i++)
    {
        for(j=0;j<no_inputs;j++)
        {
            fscanf(fdi,"%f",&w1[i][j]);
            printf("%f\t",w1[i][j]);
        }
        printf("\n");
    }
    getch();

```

```

    cleardevice();
    gotoxy(1,1);

```

```

    for(i=0;i<grossberg_nodes;i++)
    {

```

```

        for(j=0;j<kohonen_nodes;j++) {
            fscanf(fdi,"%f",&w2[i][j]);
            printf("%f\t",w2[i][j]);
        }
        printf("\n");
    }

```



```

    }
    fclose(fdi);
    outtextxy(300,400," PRESS ANY KEY TO CONTINUE... ");

    getch();
}

```

```

void save_weights()
{
    static char weightf[]="weights.wts";
    clrscr();
    FILE *fdi;
    int found = 99;
    char temp2[80];
    float sum;
    cleardevice();
    fdi=fopen(weightf,"w");
    sprintf(buffer,"SAVING FILE ==> %s \n",weightf);
    outtextxy(200,100,buffer);
    if(fdi==NULL)
    {
        printf("could not open output file!\n");
        printf("press any key to continue.");
        fclose(fdi);
        i=-999;
        return;
    }
    outtextxy(200,250,"SAVING THE WEIGHTS... ");
    // printf("\n\n\n\nSAVING THE WEIGHTS....");

    for(i=0;i<kohonen_nodes;i++)
    {
        for(j=0;j<no_inputs;j++)
            fprintf(fdi,"%f\t",w1[i][j]);
        printf("\n");
    }
    for(i=0;i<grossberg_nodes;i++)
    {
        for(j=0;j<kohonen_nodes;j++)
            fprintf(fdi,"%f\t",w2[i][j]);
        printf("\n");
    }
}

```



```
fclose(fdi);  
outtextxy(300,400," PRESS ANY KEY TO CONTINUE... ");
```

```
getch();  
}
```